

Original software publication

BlockingPy: approximate nearest neighbours for blocking of records for entity resolution

Tymoteusz Strojny^{a, b}, Maciej Beręsewicz^{c, b, *} ^a Faculty of Mathematics and Computer Science, Adam Mickiewicz University in Poznań, Uniwersytetu Poznańskiego 4, 61-614 Poznań, Poland^b Centre for the Methodology of Population Studies, Statistical Office in Poznań, Wojska Polskiego 27/29, 60-624, Poznań, Poland^c Department of Statistics, Poznań University of Economics and Business, Al. Niepodległości 10, 61-875 Poznań, Poland

ARTICLE INFO

2000 MSC:

68N01

68P10

68T10

62P25

PACS:

89.20.Ff

07.05.Kf

Keywords:

Record linkage

Deduplication

Official statistics

Economics

ABSTRACT

Entity resolution (probabilistic record linkage, deduplication) is a key step in scientific analysis and data science pipelines involving multiple data sources. The objective of entity resolution is to link records without common unique identifiers that refer to the same entity (e.g., person, company). However, without identifiers, researchers need to specify which records to compare in order to calculate matching probabilities and reduce computational complexity. One solution is to deterministically block records based on some common variables, such as names, dates of birth, or sex, or by using phonetic algorithms. However, this approach assumes that these variables are free of errors and completely observed, which is often not the case. To address this challenge, we have developed a Python package, *BlockingPy*, which relies on schema-agnostic blocking using modern approximate nearest neighbour search and graph algorithms to reduce the number of comparisons. The package supports both CPU and GPU execution. In this paper, we present the design of the package, its functionalities and two case studies related to official statistics. The presented software will be useful for researchers interested in linking data from various sources.

Metadata

Code metadata (mandatory)

Nr.	Code metadata description	Metadata
C1	Current code version	v0.2.8
C2	Permanent link to code/repository used for this code version	N/A
C3	Permanent link to Reproducible Capsule	https://blockingpy.readthedocs.io/en/latest/examples/record_linkage.html
C4	Legal Code License	MIT License
C5	Code versioning system used	Git
C6	Software code languages, tools, and services used	Python
C7	Compilation requirements, operating environments & dependencies	pandas, numpy, scipy, annoy, hnswlib, pynndescent, scikit-learn, igraph, nltk, voyager, faiss-cpu, faiss-gpu-cuvs, mlpack, model2vec
C8	If available Link to developer documentation/manual	https://blockingpy.readthedocs.io/en/latest/index.html
C9	Support email for questions	tymoteusz.strojny@gmail.com

1. Motivation and significance

Entity resolution requires statistical and computational methodologies to accurately identify matching records across datasets without unique identifiers. This process underpins countless research

endeavours in disciplines such as epidemiology, economics, official statistics and historical research where data integration is fundamental to scientific discovery [1,2]. Traditional blocking techniques, while computationally efficient, exhibit limitations when confronted with

* Corresponding author at: Department of Statistics, Poznań University of Economics and Business, Al. Niepodległości 10, 61-875 Poznań, Poland.

Email address: maciej.beresewicz@ue.poznan.pl (M. Beręsewicz).

<https://doi.org/10.1016/j.softx.2026.102583>

Received 12 October 2025; Received in revised form 21 January 2026; Accepted 23 February 2026

Available online 7 March 2026

2352-7110/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

various data quality issues such as missing values, transliteration, typographical errors, and inconsistent formatting. Existing deterministic blocking methods rely on user-defined exact-matching rules that assume key fields (e.g., name, date of birth) are error-free, potentially leading to missed links and false matches [3–5]. An alternative approach is schema-agnostic blocking [6–8], which concatenates all attribute values of a particular entity into a single textual representation before applying blocking (and can be combined with pre-trained language models).

The scientific community has developed several tools to address record linkage, including R packages such as `RecordLinkage` [9], `reclin2` [10], `fastLink` [11], or Python implementations like `splink` [12], `recordlinkage` [13], `Dedupe` [14], `pyJedAI` [15], or `FEBRL` [16]. However, these solutions typically rely on user-defined deterministic rules that may miss certain groups of records or result in large blocks of records when data irregularities are present.

Some solutions to these issues were proposed in the literature, for instance, the use of the SOUNDEX algorithm and other phonetic algorithms to block records on character strings that sound similar phonetically [3]; blocking using the Locality Sensitive Hashing (LSH; [17]), an approximate nearest neighbour (ANN) algorithm to search for similar records (see, e.g., the `k1sh` package in R) or the use of Bloom filters, useful in particular for privacy-preserving record linkage (see, [18]; for implementation see `blocklib` in Python).

The `BlockingPy` package relies on the schema-agnostic approach and uses state-of-the-art ANN search algorithms and graph-based indexing techniques, which allow it to work even with imperfect data. `BlockingPy` implements an ANN-based blocking procedure (n -gram encoding or embeddings, neighbour search, graph components) behind a single `pandas`-style call, cutting custom rule-writing and setup time.

The structure of the paper is as follows. In [Section 2](#), we briefly review the blocking procedures in existing Python packages. In [Section 3](#), we provide details regarding the implementation and the API. In [Section 4](#), we provide two examples of probabilistic record linkage and deduplication. The paper concludes with a brief review of the potential impact and conclusions.

2. Existing software covering blocking procedures

This section focuses on Python packages (`splink`, `recordlinkage`, and `blocklib`), but readers interested in deterministic blocking procedures are referred to the `fastLink` and `reclin2` packages in R. For a comparison of Python packages, see [Table A.3](#) in the Appendix.

The `splink` package allows for deterministic blocking rules using the function `block_on()` which specifies either the variables (e.g., `block_on("city")`) or more advanced combinations (e.g., `block_on("substr(first_name, 1,1)", "surname")`), which are then translated to a SQL query for the DuckDB (via the `BlockingRuleCreator` class). More advanced users may be interested in creating their own rules, which is possible through the `CustomRule` class. `Splink` allows users to specify the list of blocking rules for the `Linker` class. `Splink` also allows users to inspect the blocking via the `splink.blocking_analysis` method.

Additionally, `splink` supports blocking using phonetic algorithms such as Soundex, Metaphone, or Double Metaphone either by applying them directly within the blocking rules (e.g., `block_on("soundex(surname)")`), or by preprocessing columns to store phonetic encodings separately. In the online documentation we show that using `splink` with `BlockingPy` together results in significant improvements in the performance metrics by capturing comparison pairs that would otherwise be missed.

`recordlinkage` package supports deterministic blocking via the `recordlinkage.Index` class and its methods. This package allows for deterministic blocking using `BlockIndex()` and `SortedNeighbourhood()`

where the specification of blocking variables should be provided by a vector of column names and optionally window for the `SortedNeighbourhood()`, which accounts for multiple mistakes.

The only software that provides similar functionality is the `blocklib` package (cf. [19,20]), `vicinity` [21], and `pyJedAI`. The first uses probabilistic signatures and LSH-based Lambda-Fold Redundant as described in [19]. The second uses `semhash` [22], which allows deduplication of records using kNN-based embedding comparisons and offers exact-match blocking. The third provides an end-to-end entity resolution pipeline that includes multiple key-based blocking methods and comparison cleaning (meta-blocking) strategies, with an optional embedding-based candidate generation component based on FAISS. In contrast, `BlockingPy` allows various inputs as well as ANN algorithms and graphs, focusing on providing accurate blocking for probabilistic record linkage and deduplication. In [Appendix B](#), we compare `BlockingPy` with `blocklib` in terms of computational efficiency, accuracy, and the number of pairs from the blocking procedures. While `blocklib` is faster, it results in a significantly higher number of missed pairs and several orders of magnitude more pairs. We also provide a comparison with the GPU version of the package. In [Appendix C](#) we compare `PyJedAI` with `BlockingPy` on the Abt-Buy benchmark [23].

3. Software description

3.1. Installation

`BlockingPy` is a Python package released under the MIT license. It can be installed from PyPI using package managers such as `pip` or `Poetry`.

```
1 pip install blockingpy
2 poetry add blockingpy
```

GPU option. We also provide `blockingpy-gpu` with CUDA-enabled GPU execution:

```
1 # 1) Create env
2 mamba create -n blockingpy-gpu python=3.10 -y
3 conda activate blockingpy-gpu
4
5 # 2) Install FAISS GPU (tested cuVS nightly)
6 mamba install -c pytorch/label/nightly \
7   faiss-gpu-cuvs=1.11.0=py3.10_ha3bacd1_55_cuda12.4.0_nightly
8   -y
9
10 # 3) Install BlockingPy (GPU build)
11 pip install blockingpy-gpu
```

Notes: Import remains `import blockingpy`.

3.2. Software functionalities

This subsection describes the main functionalities provided by the `BlockingPy` package.

3.2.1. Blocking records

The core functionality of `BlockingPy` is to provide an efficient and scalable approach for blocking (also known as indexing) records in both record linkage and deduplication pipelines. Scalability is achieved mainly due to the ANN libraries that we use in our implementations. Users can input either previously computed Document-Term Matrices (DTMs) or raw text data, which the package transforms into DTMs by constructing character n -gram representations of the input. Alternatively, users can represent the input data as vector embeddings. For models supported by the `model2vec` library, users can simply pass the model identifier as a string (e.g., `"minishlab/potion-base-8m"`), and the package computes the embeddings internally. For any other embedding model, users can precompute the embeddings separately and pass them to the package as NumPy arrays.

Subsequently, similarity-based nearest neighbour search is performed for each record in either the input dataset (deduplication) or query dataset (record linkage). This step is accomplished using one of the following Python implementations of ANN algorithms:

- Exact k-Nearest Neighbours (KNN) from `mlpack` and `faiss` packages.
- Locality Sensitive Hashing (LSH) from `mlpack` [24] and `faiss` packages.
- Hierarchical Navigable Small Worlds (HNSW) from `faiss` [25], `hnswlib` [26], and `voyager` [27] packages.
- Nearest Neighbour Descent (NND) from `pynndescent` [28] package.
- Random projections and NN trees from the `annoy` [29] package.
- Inverted File Flat (IVF), IVF with Product Quantization (IVFPQ) and CAGRA [30] from `faiss` (GPU) package.

Users can fine-tune each algorithm according to their needs. Finally, `BlockingPy` groups records into blocks by identifying connected components in an undirected graph (using the `igraph` package [31]), generated by previous neighbourhood search results. In this graph, vertices represent records and edges connect pairs of records identified as neighbours. This step is necessary to handle transitive relationships (e.g., record A matches record B and record B matches record C). By using the `block()` method with the desired parameters on the `Blocker` instance, users can run the above-mentioned process to obtain blocking results.

3.2.2. Evaluation of blocking result

The blocking quality can be evaluated by providing ground truth blocks to either the `block()` or `eval()` methods. This allows users to assess the quality of the blocks computed by `BlockingPy`. The key evaluation information generated by the package is described in the sections below.

Reduction Ratio. Provides necessary details about the reduction in comparison pairs if the given blocks are applied to a further record linkage or deduplication procedure. For deduplication:

$$RR_{\text{deduplication}} = 1 - \frac{\sum_{i=1}^k \binom{|B_i|}{2}}{\binom{n}{2}},$$

where k is the total number of blocks, n is the total number of records in the dataset, and $|B_i|$ is the number of records in the i -th block. $\sum_{i=1}^k \binom{|B_i|}{2}$ is the number of comparisons after blocking, while $\binom{n}{2}$ is the total number of possible comparisons without blocking. For record linkage the reduction ratio is defined as follows

$$RR_{\text{record_linkage}} = 1 - \frac{\sum_{i=1}^k |B_{i,x}| \cdot |B_{i,y}|}{(m \cdot n)},$$

where m and n are the sizes of datasets X and Y , and k is the total number of blocks. The term $|B_{i,x}|$ is the number of unique records from dataset X in the i -th block, while $|B_{i,y}|$ is the number of unique records from dataset Y in the i -th block. The expression $\sum_{i=1}^k |B_{i,x}| \cdot |B_{i,y}|$ represents the number of comparisons after blocking.

Confusion matrix presents results in comparison to ground-truth blocks in a pairwise manner (e.g., one true positive pair occurs when both records from the comparison pair belong to the same predicted `block` and to the same ground-truth `block` in the evaluation data frame).

- True Positive (TP): Record pairs correctly matched within the same block.
- False Positive (FP): Record pairs identified as matches that are not true matches within the same block.

Table 1
Evaluation metrics.

Metric	Formula	Metric	Formula
Recall (Pairs Completeness)	$\frac{TP}{TP+FN}$	Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$
Precision (Pairs Quality)	$\frac{TP}{TP+FP}$	Specificity	$\frac{TN}{TN+FP}$
F1 Score	$2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$	False Positive Rate	$\frac{FP}{FP+TN}$
False Negative Rate	$\frac{FN}{FN+TP}$		

- True Negative (TN): Record pairs correctly identified as non-matches (different blocks).
- False Negative (FN): Record pairs identified as non-matches that are actually true matches within the same block.

Finally, [Table 1](#) presents metrics that enable users to evaluate blocking procedures.

3.3. Software architecture

The architecture of `BlockingPy` provides a modular design, where each component is responsible for a specific aspect of the entire blocking workflow. This approach facilitates the integration of new ANN algorithms and modifications to existing functionality. The main components are illustrated in [Fig. 1](#), and their roles are described below.

Blocker. The primary component of the `BlockingPy` architecture. It coordinates interactions among other components, managing the entire workflow of the package from handling data ingestion to generating results.

ANN Algorithms. This is a collection of ANN implementations with a common interface for the straightforward addition of new algorithms. The parameters of each algorithm can be adjusted by providing the `Controls` dictionary to the `control_ann` parameter within the `block` method. An example of this is shown in [Listing 1](#). All ANN backends share a single interface across CPU and GPU.

Input Processing. Manages the data pre-processing and transformation across multiple input formats. Converts dense matrices, sparse matrices, and raw text data to Numpy arrays or Scipy CSR matrices via the `DataHandler` class for efficient processing. Currently, we support transformations based on n -grams and vector embeddings.

Controls. Manages the configuration system for both text and ANN parameters. Includes customization for algorithm-specific options e.g., metric selection, search and query parameters, alongside tokenization rules and n -gram size, or alternatively the choice of an embedding model. [Listing 2](#) presents an example of a `Controls` dictionary for the `hnsw` algorithm.

Input Validation. Ensures the validity of the data and parameters while providing error messages for the user.

Blocking Result. The component responsible for managing results and the corresponding information about the blocking process e.g., algorithm used, evaluation metrics, reduction ratio, and block distribution.

4. Illustrative examples

In the following section, we demonstrate the `BlockingPy` package's functionality using two examples: one for record linkage and one for deduplication. We aim to show the ease of use and effectiveness of the package in blocking records for entity resolution.

4.1. Record linkage

In this example, we use the `Cis-Census` datasets (for more information see [32]) to demonstrate the record linkage capabilities of `BlockingPy`. These datasets contain fictitious personal information, with the `census` dataset comprising 25,343 records and the `cis` (Customer Information System) dataset containing 24,613 records. [Table 2](#) shows sample records from the `census` dataset after the initial preparation described in [Listing 3](#). The `cis` dataset is structured similarly.

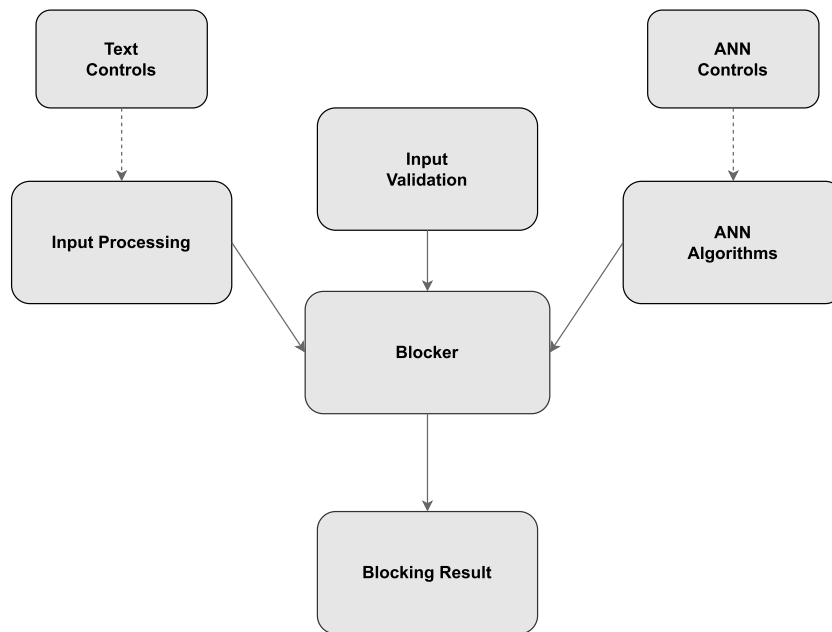


Fig. 1. The Architecture of the BlockingPy package.

```

1 blocker = Blocker()
2 result = blocker.block(x=x, ann='voyager', control_ann=
  control_ann)

```

Listing 1. Setting the algorithm and fine-tuning with control_ann parameter.

```

1 control_ann = {
2     'random_seed': 2025,
3     'hnsu': {
4         'distance': 'cosine',
5         'k_search': 30,
6         'n_threads': 1,
7         'path': None,
8         'M': 25,
9         'ef_c': 200,
10        'ef_s': 200,
11    }
12 }

```

Listing 2. Controls for the hnsu algorithm.

Table 2

Example records from the census dataset with variables used for the txt column. Variables (changed here for better printing): NAME1 (first name), NAME2 (surname), SEX (gender), D, M, Y (birth date), ENUMCAP (address), ENUMPC (postal code).

NAME1	NAME2	SEX	D	M	Y	ENUMCAP	ENUMPC
COUIE	PRICE	M	1	6	1960	1 WINDSOR ROAD	DE03US
ABBIE	PVICE	F	9	11	1961	1 WINDSOR ROAD	DE03US
LACEY	PRICE	F	7	2	1999	1 WINDSOR ROAD	DE03US
SAMUEL	PRICE	M	13	4	1990	1 WINDSOR ROAD	DE03US
JOSEPH	PRICE	M	20	4	1986	1 WINDSOR ROAD	DE03US

Before blocking, we merge all fields of interest (without ID) into a single field (i.e., txt) that we will pass to the algorithm as shown in Listing 4 (i.e., we use *schema-agnostic* blocking). This step is performed for both datasets.

Subsequently, in Listing 5 we initialise Blocker, invoke the block() method, and select the hnsu algorithm with default parameters to obtain the results. The tokenization and white-space removal are handled by the algorithm internally and can be modified via the control_txt

```

1 from blockingpy import Blocker
2 import pandas as pd
3 from blockingpy.datasets import load_census_cis_data
4
5 census, cis = load_census_cis_data()
6
7 census = census[["PERSON_ID", "PERNAME1", "PERNAME2", "SEX", "
8     DOB_DAY", "DOB_MON", "DOB_YEAR", "ENUMCAP", "ENUMPC"]]
9 cis = cis[["PERSON_ID", "PERNAME1", "PERNAME2", "SEX", "DOB_DAY"
10     , "DOB_MON", "DOB_YEAR", "ENUMCAP", "ENUMPC"]]

```

Listing 3. Preprocessing the record linkage example data.

```

1 for df in [census, cis]:
2     df[['DOB_DAY', 'DOB_MON', 'DOB_YEAR']] = (
3         df[['DOB_DAY', 'DOB_MON', 'DOB_YEAR']]
4         .astype("Int64")
5         .astype(str)
6         .replace('<NA>', '')
7     )
8     df.fillna('', inplace=True)
9
10    df['txt'] = (
11        df['PERNAME1']
12        + df['PERNAME2']
13        + df['SEX']
14        + df['DOB_DAY']
15        + df['DOB_MON']
16        + df['DOB_YEAR']
17        + df['ENUMCAP']
18        + df['ENUMPC']
19    )

```

Listing 4. Creating the txt column.

```

1 blocker = Blocker()
2 rec_lin_result = blocker.block(
3     x=census['txt'],
4     y=cis['txt'],
5     ann='hnsu',
6     random_seed=42
7 )

```

Listing 5. Record linkage blocking example.

parameter. Additional algorithm-specific parameters can be fine-tuned via the `control_ann` argument.

The output is a data frame containing rows with indices from both datasets (x , y) alongside the assigned block (an integer starting at 0) and the distance computed using one of the available metrics. Listing 6 presents a snippet of the output in this example. Listing 7 shows the basic blocking information logged by the package. The first section provides details about the algorithm used, the number of generated blocks, the number of columns used for blocking (created by shingling, a preprocessing step that converts strings into overlapping letter groups used as input to the ANN algorithm), and the reduction ratio. The second section displays the distribution of block sizes; for example, there were 591 blocks created, each containing 3 records.

We observe a reduction ratio of 0.9999, which indicates that the possible number of candidate pairs is substantially reduced, thus

lowering the computational complexity, which is crucial for large-scale datasets.

With a unique personal identifier available in both datasets, we can evaluate the algorithm. For that, we create a `true_blocks` data frame which represents indices from both datasets and their true block, as shown in Listing 8. In this case, every record in each block should point to the same unique entity. Furthermore, in Listing 9 we pass the `rec_lin_result` object (from Listing 5) and the `true_blocks` to the `eval` method to obtain the evaluation results. In this example, we randomly sampled 1000 ground-truth pairs to showcase the evaluation functionality of the `BlockingPy` package.

Through this step, the user can access both the evaluation metrics and the confusion matrix. Evaluation is performed only for records that are available in `true_blocks`. We assume that we have no knowledge about the records not included in the ground-truth data frame. Listings 10

```

1 print(rec_lin_result.result.head())
2      x  y  block      dist
3 0 17339  0      0 0.134151
4 1  9567  1      1 0.064307
5 2 10389  2      2 0.044183
6 3 24258  3      3 0.182125
7 4  3714  4      4 0.288487

```

Listing 6. Record linkage example of blocking result.

```

1 print(rec_lin_result)
2 =====
3 Blocking based on the hnsu method.
4 Number of blocks: 23993
5 Number of columns created for blocking: 1072
6 Reduction ratio: 0.999961
7 =====
8 Distribution of the size of the blocks:
9 Block Size | Number of Blocks
10          2 | 23388
11          3 | 591
12          4 | 13
13          5 | 1

```

Listing 7. Basic blocking information.

```

1 census['x'] = range(len(census))
2 cis['y'] = range(len(cis))
3
4 true_blocks = pd.merge(
5     left=census[['PERSON_ID', 'x']],
6     right=cis[['PERSON_ID', 'y']],
7     on='PERSON_ID'
8 )
9
10 true_blocks['block'] = range(len(true_blocks))

```

Listing 8. Preparation of true_blocks.

```

1 true_blocks = true_blocks.sample(1000, random_state=42)
2 eval_result = blocker.eval(rec_lin_result, true_blocks[['x',
3     'y', 'block']])

```

Listing 9. Record Linkage Evaluation Code.

```

1 print(eval_result.metrics)
2 recall      0.997000
3 precision    1.000000
4 fpr          0.000000
5 fnr          0.003000
6 accuracy     0.999997
7 specificity   1.000000
8 f1_score     0.998498

```

Listing 10. Evaluation Metrics.

and 11 present the metrics and confusion matrix computed in this example.

The results present BlockingPy's ability to efficiently and accurately block records for record linkage tasks. The full version of this example is available on the documentation website.

4.2. Deduplication

In this example, our aim is to show how BlockingPy can be integrated into full entity resolution workflows. For that, we will present the deduplication of the febr11 dataset obtained from the

recordlinkage package [13]. This dataset was generated using the FEBRL software and contains 1000 records with fictitious personal information, of which 500 are original and 500 are duplicates with introduced errors. Firstly, we prepare the blocking key by imputing missing values with an empty string and concatenating fields with personal information into a single field, which we will use for blocking. After initialising Blocker and performing blocking on said data, we are given the blocks by BlockingPy. Listing 12 presents this process, and the Listing 13 shows the blocking information output and the first five rows of the actual result.

```

1 print(eval_result.confusion)
2
3 Predicted Positive Predicted Negative
4 Actual Positive      997              3
5 Actual Negative      0             999000

```

Listing 11. Confusion Matrix.

```

1 import recordlinkage
2 from recordlinkage.datasets import load_febr11
3 from blockingpy import Blocker
4 import pandas as pd
5 import numpy as np
6
7 df = load_febr11()
8
9 df = df.fillna('')
10 df['txt'] = df['given_name'] + df['surname'] + \
11             df['street_number'] + df['address_1'] + \
12             df['address_2'] + df['suburb'] + \
13             df['postcode'] + df['state'] + \
14             df['date_of_birth'] + df['soc_sec_id']
15
16 blocker = Blocker()
17 blocking_result = blocker.block(
18     x=df['txt'],
19     ann='hnsu',
20     random_seed=42
21 )

```

Listing 12. Preprocessing and blocking.

```

1 print(blocking_result)
2 =====
3 Blocking based on the hnsu method.
4 Number of blocks: 500
5 Number of columns used for blocking: 1023
6 Reduction ratio: 0.998999
7 =====
8 Distribution of the size of the blocks:
9 Block Size | Number of Blocks
10           2 | 500
11
12 print(blocking_result.result.head())
13
14   x  y  block  dist
15  474 0     0  0.048375
16  330 1     1  0.038961
17  351 2     2  0.086690
18  290 3     3  0.024617
19  333 4     4  0.105662

```

Listing 13. Blocking Information.


```

1 df['id'] = range(len(df))
2 df_final = blocking_result.add_block_column(df, id_col_left='
  id')

```

Listing 14. Creating the block column.

```

1 indexer = recordlinkage.Index()
2 indexer.block('block')
3 pairs = indexer.index(df_final)

```

Listing 15. Integration with recordlinkage.

Both columns *x* and *y* refer to indices from the febr11 dataset alongside their designated *block* and the distance measured between them using one of the available metrics (cosine in this example, the default for *hsw*). Furthermore, the necessary step to make integration with *recordlinkage* possible is to add the *block* column to the original data frame, which we can do with the *add_block_column* method as presented in Listing 14.

After calling the *block* method on the *Index* with the previously obtained *block* column in Listing 15 we can proceed with the usual *recordlinkage* workflow.

The full version of this example and many others, alongside the code, can be found in the documentation.

5. Impact

The proposed approach is used at Statistics Poland in the procedure for the linkage of records between administrative datasets as well as within one dataset. For instance, we have used this approach to deduplicate records of forced migrants crossing the Polish-Ukrainian border after the full-scale invasion of Ukraine by Russia on February 24, 2022. We also used this technique to determine the number of residents of Ukraine under temporary protection in Poland as of March 31, 2023 [33]. The main motivation behind this was the problem of the lack of identifiers as well as the transliteration from Ukrainian/Russian to Polish language (e.g., Олександр, should be Ołeksandr not Aleksandr). Before implementing this solution, Statistics Poland relied on user-defined rules for linking data without identifiers, which often resulted in imprecise population flow estimates due to missed matches or false positives.

We believe that the proposed software will be of interest to researchers and practitioners who link data without identifiers or would like to apply large language models to detect duplicates but need to reduce costs by providing a small number of comparison buckets.

6. Conclusions

The current project is designed to be extensible so that it can accommodate new ANN algorithms and new input data formats. Any contributions are warmly welcomed; see <https://github.com/ncn-foreigners/BlockingPy/issues> for a feature request and bug tracker.

Future updates will focus on adding functionality for privacy-preserving record linkage without directly sharing personally identifiable information.

CRedit authorship contribution statement

Tymoteusz Strojny: Writing – review & editing, Writing – original draft, Software, Conceptualization. **Maciej Beręsewicz:** Writing – review & editing, Writing – original draft, Validation, Supervision, Project administration, Methodology, Conceptualization.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used DeepL and Claude.ai to proofread the paper. After using this the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We thank the reviewers for their comments on this manuscript. This software was a part of Tymoteusz Strojny's Bachelor's thesis at Poznań University of Economics and Business, Poland. *BlockingPy* is based on the *blocking* package in R (see <https://ncn-foreigners.github.io/blocking/>).

Work on this package is supported by the National Science Centre, OPUS 20 grant no. 2020/39/B/HS4/00941 (*Towards census-like statistics for foreign-born populations – quality, data integration and estimation*).

This software was peer-reviewed and accepted by the pyOpenSci organisation (see <https://github.com/pyOpenSci/software-submission/issues/232>).

Appendix A. Comparison of blocking methods implemented in splink, blocklib, recordLinkage, pyJedi and BlockingPy

Table A.3

Comparison of blocking methods implemented in selected software.

Package	Blocking methods
splink	Rule-based / deterministic blocking (SQL-style rules), phonetic blocking (e.g., Soundex, Metaphone).
blocklib	Probabilistic signature blocking (p-sig), LSH-based λ -fold redundant blocking (Bloom filter + LSH).
recordlinkage	Exact blocking, Sorted Neighbourhood, full indexing (all pairs), random indexing (sampled pairs).
pyJediAI	Token Blocking, Sorted Neighbourhood, Extended Sorted Neighbourhood, Q-Grams Blocking, Extended Q-Grams Blocking, Suffix Arrays Blocking, Extended Suffix Arrays Blocking, Embeddings-based NN blocking.
BlockingPy	ANN-based blocking via FAISS, HNSW (faiss / hnswlib / voyager), NN-Descent (pynndescent), Annoy, plus GPU indices (e.g., IVF, IVFPQ, CAGRA); blocks from connected components of the NN graph.

Appendix B. Numerical comparison with blocklib

We compare *BlockingPy* and *blocklib* in three synthetic datasets (1,500, 15,000, and 150,000 records) with duplicates 500, 5,000, and

Table B.4
CPU results on three datasets.

Algorithm	Size	Time [s]	Recall	RR	Pairs (M)
P-Sig	1500	0.051 (0.010)	0.599 (0.000)	0.996 (0.000)	0.004 (0.000)
λ -fold LSH	1500	0.193 (0.009)	0.466 (0.033)	0.991 (0.003)	0.010 (0.003)
BlockingPy (voyager)	1500	0.293 (0.029)	0.952 (0.006)	0.997 (0.000)	0.003 (0.000)
BlockingPy (faiss_hnsw)	1500	0.164 (0.049)	0.960 (0.000)	0.998 (0.000)	0.003 (0.000)
BlockingPy (faiss_lsh)	1500	0.185 (0.015)	0.955 (0.008)	0.997 (0.000)	0.003 (0.000)
P-Sig	15,000	0.388 (0.087)	0.616 (0.000)	0.996 (0.000)	0.407 (0.000)
λ -fold LSH	15,000	2.016 (0.251)	0.454 (0.028)	0.992 (0.003)	0.940 (0.337)
BlockingPy (voyager)	15,000	6.777 (0.903)	0.876 (0.005)	1.000 (0.000)	0.041 (0.001)
BlockingPy (faiss_hnsw)	15,000	12.029 (1.681)	0.913 (0.000)	1.000 (0.000)	0.031 (0.000)
BlockingPy (faiss_lsh)	15,000	1.398 (0.148)	0.900 (0.004)	1.000 (0.000)	0.033 (0.001)
P-Sig	150,000	3.444 (0.091)	0.609 (0.000)	0.996 (0.000)	40.231 (0.000)
λ -fold LSH	150,000	19.848 (0.476)	0.450 (0.026)	0.992 (0.003)	95.064 (34.252)
BlockingPy (voyager)	150,000	110.341 (3.815)	0.715 (0.005)	1.000 (0.000)	0.675 (0.021)
BlockingPy (faiss_hnsw)	150,000	250.012 (4.232)	0.832 (0.001)	1.000 (0.000)	0.375 (0.001)
BlockingPy (faiss_lsh)	150,000	56.544 (1.315)	0.818 (0.001)	1.000 (0.000)	0.400 (0.005)

Table B.5
GPU results on 150,000 records dataset.

Algorithm	Size	Time [s]	Recall	RR	Pairs (M)
BlockingPy (gpu_faiss cagra)	150,000	51.084 (0.802)	0.827 (0.000)	1.000 (0.000)	0.380 (0.000)
BlockingPy (gpu_faiss flat)	150,000	22.511 (0.508)	0.839 (0.000)	1.000 (0.000)	0.367 (0.000)
BlockingPy (gpu_faiss ivf)	150,000	71.458 (2.185)	0.801 (0.004)	1.000 (0.000)	0.487 (0.021)

50,000, respectively. The results are averaged over 10 runs. The datasets were generated with `geco3` and contain fictional personal information (given name, middle name, surname, municipality, date of birth, and nationality). CPU experiments used a 6-core Intel Core i5 with 16 GB RAM and Python 3.12. GPU runs used an NVIDIA RTX 3050 (4 GB VRAM) and Python 3.10. We report time (s), recall (Pairs Completeness), reduction ratio (RR), and candidate pairs (millions). The comparison is also available in the documentation along with the code and data in the project repository. Preprocessing: for BlockingPy, we concatenate all fields of interest into a single text field; for `blocklib`, we tune algorithm parameters to approximately match the reduction ratio of the other methods to ease comparison (Tables B.4 and B.5)

Appendix C. Comparison with PyJedAI

We compare BlockingPy and PyJedAI on the Abt-Buy clean-clean entity resolution benchmark, which consists of two product catalogs from the online retailers [Abt.com](#) and [Buy.com](#). We use the Abt-Buy dataset as provided in the pyJedAI repository, which contains 1076 records in each source and a gold-standard mapping between the two sources. The hardware setup is the same as in Appendix B; the experiments were run with Python 3.11 (with PyJedAI using NumPy 2.4.1 and BlockingPy using NumPy 1.26.4). We report runtime (s), recall, precision, F_1 , and the number of candidate comparison pairs.

For BlockingPy, we use the HNSW backend with the `minishlab/potion-base-32M` embedding model; all remaining parameters are left at their defaults. For PyJedAI, we use the `sminilm`

vectorizer with $k = 5$ nearest neighbours and Euclidean distance. The data (and an example workflow) are available in the pyJedAI documentation/repository. The BlockingPy workflow used for this comparison is likewise available in the BlockingPy repository (Table C.6).

Data availability

The datasets used in Section 4 are included with the BlockingPy package and are also publicly available from their original sources.

References

- [1] Binette O, Steorts RC. (Almost) all of entity resolution. *Science Advances* 2022;8(12):eabi8021. <https://doi.org/10.1126/sciadv.abi8021>. <https://www.science.org/doi/pdf/10.1126/sciadv.abi8021>. <https://www.science.org/doi/abs/10.1126/sciadv.abi8021>.
- [2] Tancredi A, Steorts R, Liseo B. A unified framework for de-duplication and population size estimation (with discussion). *Bayesian Anal* 2020;15(2):633–82. <https://doi.org/10.1214/19-BA1146>
- [3] Christen P. Data matching: concepts and techniques for record linkage, entity resolution, and duplicate detection, *Data-Centric Systems and Applications*, Springer; 2012. <https://doi.org/10.1007/978-3-642-31164-2>
- [4] Herzog TN, Scheuren FJ, Winkler WE. Data quality and record linkage techniques. Springer; 2007. <https://doi.org/10.1007/0-387-69505-2>
- [5] Jugl M, Kirsten T. Gecko: a Python library for the generation and mutation of realistic personal identification data at scale. *SoftwareX* 2024;27:101846. <https://doi.org/10.1016/j.swx.2024.101846>
- [6] Papadakis G, Alexiou G, Papastefanatos G, Koutrika G. Schema-agnostic VS schema-based configurations for blocking methods on homogeneous data. *PVLDB* 2015;9(4):312–23.
- [7] Thirumuruganathan S, Tang N, Ouzzani M, Doan A. Deep learning for blocking in entity matching: a design space exploration. *PVLDB* 2021;14(11):2459–72.
- [8] Papadakis G, Skoutas D, Thanos E, Palpanas T. Blocking and filtering techniques for entity resolution: a survey. *ACM Comput Surv* 2020;53(2):31:1–31:42. <https://doi.org/10.1145/3377455>
- [9] Sariyar M, Borg A. The RecordLinkage package: detecting errors in data. *The R J* 2010;2(2):61–7. <https://doi.org/10.32614/RJ-2010-017>
- [10] van der Laan DJ. reclin2: a toolkit for record linkage and deduplication. *R J* 2022;14(2).
- [11] Enamorado T, Fifield B, Imai K. Using a probabilistic model to assist merging of large-scale administrative records. *Am Polit Sci Rev* 2019;113(2):353–71. <https://doi.org/10.1017/S0003055418000783>
- [12] Linacre R, Lindsay S, Manassis T, Slade Z, Hepworth T, Kennedy R, Bond A. Splink: free software for probabilistic record linkage at scale. *Int J Popul Data Sci Aug* 2022;7(3). <https://doi.org/10.23889/ijpds.v7i3.1794>. <https://ijpds.org/article/view/1794>.
- [13] de Bruin J, Anderson J, Becker J, Wong H, tylerbinski, Waleń T, Elias D, Weytjens J, GG D, Knuth T, Varner H, Baldassini L, Hoffman M, Antoine M, Norton R, Teigman T, Deplasse V, tylerbinski. J535d165/recordlinkage: v0.16. Jul, 2023. <https://doi.org/10.5281/zenodo.8169000>

Table C.6

Comparison on the Abt-Buy benchmark.

Algorithm	Time [s]	Recall	Precision	F_1	Pairs
PyJedAI	52.6	0.9377	0.1875	0.3126	5380
BlockingPy	4.4	0.8234	0.8234	0.8234	1076

Note: The identical recall, precision, and F_1 values reported here are expected in this setting and are not a reporting error: in BlockingPy we use $k = 1$ (one candidate per record) and, for this experiment, the graph-based step did not find additional links, so evaluation is effectively performed on the same one-to-one pair set; under these conditions these three metrics coincide.

- [14] Gregg F, Eder D. dedupe Jan 2022. <https://github.com/dedupeio/dedupe>.
- [15] Nikolettos K, Papadakis G, Koubarakis M. pyjedai: a lightsaber for link discovery. In: Dimou A, Haller A, Gentile AL, Ristoski P, editors. Proceedings of the ISWC 2022 posters, demos and industry tracks: from novel ideas to industrial practice co-located with 21st international semantic web conference (ISWC 2022), virtual conference, hangzhou, china, October 23–27, 2022 of CEUR workshop proceedings, vol. 3254. CEUR-WS.org; 2022. p. 328–32. <https://ceur-ws.org/Vol-3254/paper366.pdf>.
- [16] Christen P. Febrl—an open source data cleaning, deduplication and record linkage system with a graphical user interface. In: Proceedings of the 14th ACM SIGKDD international conference on knowledge discovery and data mining; 2008. p. 1065–8.
- [17] Steorts R. klsh: blocking for record linkage, R package version 0.1.0 2020. <https://CRAN.R-project.org/package=klsh>.
- [18] Christen P, Ranbaduge T, Schnell R. Linking sensitive data: methods and techniques for practical Privacy-Preserving information sharing. Cham: Springer; 2020. <https://doi.org/10.1007/978-3-030-59706-1>
- [19] Zhang Y, Ng KS, Churchill T, Christen P. Scalable entity resolution using probabilistic signatures on parallel databases. In: Proceedings of the 27th ACM international conference on information and knowledge management, CIKM '18. New York, NY, USA: Association for Computing Machinery; 2018. p. 2213–21. <https://doi.org/10.1145/3269206.3272016>
- [20] Data61 C. Anonlink private record linkage system; 2017. <https://github.com/data61/anonlink>.
- [21] MinishLab. Vicinity: lightweight nearest neighbors with flexible backends; 2025. <https://github.com/MinishLab/vicinity>, version 0.4.1.
- [22] van Dongen T, Tulkens S. SemHash: fast semantic text deduplication; 2025. <https://github.com/MinishLab/semhash>, gitHub repository.
- [23] Köpcke H, Thor A, Rahm E. Evaluation of entity resolution approaches on real-world match problems. Proc VLDB Endow 2010;3(1–2):484–93. <https://doi.org/10.14778/1920841.1920904>
- [24] Curtin RR, Edel M, Shrit O, Agrawal S, Basak S, Balamuta JJ, Birmingham R, Dutt K, Edelbuettel D, Garg R, Jaiswal S, Kaushik A, Kim S, Mukherjee A, Sai NG, Sharma N, Parihar YS, Swain R, Sanderson C. Mlpack 4: a fast, header-only c++ machine learning library. J Open Source Softw 2023;8(82):5026. <https://doi.org/10.21105/joss.05026>
- [25] Douze M, Guzhva A, Deng C, Johnson J, Szilvassy G, Mazaré P-E, Lomeli M, Hosseini L, Jégou H. The faiss library. arXiv:2401.08281 [arXiv preprint]. 2025.
- [26] Malkov YA, Yashunin DA. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Trans Pattern Anal Mach Intell 2018;42(4):824–36.
- [27] Spotify. Voyager: approximate nearest neighbors library; 2023. <https://github.com/spotify/voyager> [Accessed: 22 January 2025].
- [28] McInnes L. Pynndescent: a Python library for approximate nearest neighbors; 2019. <https://github.com/lmcinnes/pynndescent> [Accessed: 22 January 2025].
- [29] Spotify. Annoy (approximate nearest neighbors OH yeah); 2017. <https://github.com/spotify/annoy> [Accessed: 22 January 2025].
- [30] Ootomo H, Naruse A, Nolet C, Wang R, Feher T, Wang Y. Cagra: highly parallel graph construction and approximate nearest neighbor search for gpus. In: 2024 IEEE 40th international conference on data engineering (ICDE). Utrecht, Netherlands: IEEE; 2024. p. 4236–47.
- [31] Csardi G, Nepusz T. The igraph software. Complex Syst 2006;1695:1–9.
- [32] McLeod P, Heasman D, Forbes I. Census and CIS datasets, Retrieved from Internet Archive: 2011. https://wayback.archive-it.org/12090/20231221144450/https://cros-legacy.ec.europa.eu/content/job-training_en, dataset created for the ESSnet DI on-the-job training course, Southampton, 25–28 January 2011.
- [33] Statistics Poland. Residents of Ukraine under temporary protection, News release, Statistics Poland, data as at 31 March 2023 Dec 2023. <https://stat.gov.pl/en/topics/population/international-migration/residents-of-ukraine-under-temporary-protection,9,1.html>.